

Resumo de Tópicos Contextuais

do Claude Code

Comandos, modos avançados, contexto/CLAUDE.md, configuração com /init, plugins/MCP e prompts eficazes — seis módulos de fundamentos consolidados nesta sessão de trabalho.

■ Comandos e Atalhos

■ Outros Modos de Operação

■ Contexto & CLAUDE.md

■ /init

■ Plugins e MCP

■ Prompts Eficazes

O que foi coberto nesta sessão

Este documento reúne, em formato executivo, os seis módulos do Claude Code estudados nesta sessão — cada um com um PDF técnico completo já salvo na respectiva subpasta. Aqui você encontra o essencial de cada um, pronto para consulta rápida.

Comandos e Atalhos

A referência rápida do Claude Code CLI: comandos de barra, modos via Shift+Tab e atalhos de teclado.

Outros Modos de Operação

Checkpoint, Rewind, Yolo Mode e Ralph Loop — segurança, velocidade e automação de longa duração.

Contexto, Memória e CLAUDE.md

O que sobrevive ao fechar o terminal e como escrever um CLAUDE.md eficaz.

Configuração do Projeto com /init

Do zero ao ambiente configurado: CLAUDE.md, settings.json e comandos customizados.

Plugins e MCP

Model Context Protocol: conectando o Claude a bancos de dados, APIs e ferramentas externas.

Escrevendo Prompts Eficazes

Objetivo, contexto, restrições, formato e exemplos — a anatomia do prompt que gera código correto de primeira.

01

Comandos e Atalhos

A referência rápida do Claude Code CLI

Base de toda a formação: os comandos de barra mais usados, os quatro modos de operação alternados com Shift+Tab e os atalhos de teclado que aceleram o dia a dia no terminal.

AJUDA /help — lista completa de comandos	MODELO /model — Opus 4.8 · Sonnet 5 · Haiku 4.5
ESTADO DA SESSÃO /status e /context	REINÍCIO LIMPO /clear apaga histórico e contexto
CONSUMO /stats — tokens e custo estimado	MODOS (SHIFT+TAB) Auto · Plan · Manual · Bypass

CHEAT SHEET

Shift+Tab # alterna Auto / Plan / Manual / Bypass

Ctrl+C # interrompe a execução atual

Ctrl+D # encerra a sessão (= /exit)

Ctrl+R # busca no histórico de comandos

- **/context + /clear** ao trocar de tarefa: contexto limpo gera respostas melhores e mais baratas.
- Use **/model** por tipo de tarefa — Opus para arquitetura, Sonnet para desenvolvimento geral, Haiku para tarefas simples.
- Ative o modo **Plan** antes de refatorações críticas: o Claude planeja e aguarda aprovação antes de tocar em arquivos.

02

Outros Modos de Operação

Checkpoint, Rewind, Yolo e Ralph Loop

Checkpoint e Rewind formam o par "save/undo" do projeto; Yolo Mode remove as confirmações passo a passo para ganhar velocidade; Ralph Loop executa um plugin em ciclos até atingir um objetivo.

CHECKPOINT snapshot do projeto — o "save point"	REWIND restaura o projeto a um checkpoint
YOLO MODE executa tudo sem pedir confirmação	RALPH LOOP itera um plugin até o objetivo ser atingido

COMANDOS

```
/checkpoint      # nome automático
/checkpoint my-save  # nome customizado
/rewind          # volta ao checkpoint mais recente
/rewind my-save    # volta a um checkpoint específico
/ralph-loop:[plugin] "objetivo em linguagem natural"
```

- Checkpoint e Rewind **andam juntos**: sem checkpoint criado, o /rewind não tem para onde voltar.
- Sempre crie um **checkpoint antes do Yolo Mode** — operações destrutivas rodam sem qualquer confirmação.
- **Ralph Loop** é para automação de longa duração; Yolo é para velocidade pontual.

03

Contexto, Memória e CLAUDE.md

O que sobrevive ao fechar o terminal

O Claude não tem memória nativa entre sessões — cada terminal novo começa do zero. O CLAUDE.md na raiz do projeto, lido automaticamente, é o contrato que dá a ele uma memória persistente e controlável.

PERSISTENTE CLAUDE.md — stack, padrões, regras	ATIVO arquivos abertos na sessão
SESSÃO histórico do diálogo atual	CONFIGURAÇÃO settings.json — ferramentas, permissões
ORDEM DE LEITURA ./CLAUDE.md → ~/.claude/CLAUDE.md → ./src/CLAUDE.md	SOBREVIVE AO FECHAR só CLAUDE.md e settings.json

CLAUDE.MD — EXEMPLO

```
# Projeto: E-commerce API
## Stack
- Python 3.11 + FastAPI
- PostgreSQL 15 + SQLAlchemy
## Regras importantes
- NUNCA alterar models.py sem avisar
```

- Seja **específico**: "Python 3.11 + FastAPI" vale mais que "uso Python".
- **Nunca** coloque segredos (senhas, tokens) no CLAUDE.md.
- "O CLAUDE.md é o contrato": quanto mais rico, melhor o resultado nas sessões seguintes.

04

Configuração do Projeto com /init

Do zero ao ambiente configurado

/init varre o código-fonte e as configs do projeto para detectar stack e frameworks, gera um CLAUDE.md pré-preenchido e monta a pasta .claude/ com permissões e comandos customizáveis.

GERA CLAUDE.md + .claude/settings.json + commands/	DETECTA Node/React · Python · Go · Monorepo
PERMISSÕES allow / deny com glob patterns	PRIORIDADE deny sempre vence allow
COMANDOS CUSTOM cada .md em commands/ vira um /comando	ENV variáveis de sessão — nunca segredos reais

```
.CLAUDE/SETTINGS.JSON

{
  "permissions": {
    "allow": ["Bash(npm run *)", "Read(**)", "Write(src/**)"],
    "deny": ["Bash(rm -rf *)", "Write(.env)"]
  }
}
```

- **Deny sempre vence allow** — um deny em Bash(git push *) bloqueia o push mesmo com Bash(git *) liberado.
- Sempre revise e complete o CLAUDE.md gerado; versione a pasta .claude/ junto com o projeto.

05

Plugins e MCP

Model Context Protocol

MCP é um padrão aberto que conecta o Claude a ferramentas e dados externos via JSON-RPC — bancos de dados, APIs, repositórios — com acesso em tempo real, não apenas ao conhecimento de treinamento.

COMANDOS /plugin list · install · remove · info	TRANSPORTES stdio · HTTP/SSE · WebSocket
SERVIDORES POPULARES PostgreSQL · GitHub · Slack · Filesystem	CONFIGURAÇÃO bloco mcpServers em settings.json

SETTINGS.JSON — MCPSERVERS

```
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["@mcp/server-postgres"],
      "env": { "DATABASE_URL": "${DB_URL}" }
    }
  }
}
```

- Credenciais sempre em **variáveis de ambiente** — nunca escritas direto no settings.json.
- Bancos de produção: usuário **READ ONLY** (só SELECT), princípio do menor privilégio.

06

Escrevendo Prompts Eficazes

Prompt preciso = código perfeito

"O Claude é tão bom quanto o que você pede." A qualidade do código gerado começa na qualidade do prompt — objetivo, contexto, restrições, formato e exemplos.

OBJETIVO obrigatório — verbo de ação: crie/refatore/analise	CONTEXTO obrigatório — stack, versões, padrão a seguir
RESTRIÇÕES opcional — o que NÃO fazer, escopo	FORMATO opcional — retorno, docstring, testes
EXEMPLOS few-shot > descrição longa	TÉCNICAS Chain of Thought · Decomposição · Role prompting

VAGO VS. PRECISO

// vago

"Faça uma função de login"

// preciso

"login(email, password) em Python 3.11: valide email com regex, use bcrypt, retorne Dict com {token, user_id} ou lance AuthError"

- O Claude **não sabe nada do seu projeto** além do que você conta — caminhos, versões e padrões são obrigatórios.
- **Diga o que NÃO fazer**: escopo de arquivo, interface pública, dependências e testes a preservar.